
Bridging Offline and Online DPO Through On-Policy Data Mixing

Issey Sone

Department of Computer and Mathematical Science
University of Toronto Scarborough
Scarborough, ON M1C 1A4
issey.sone@mail.utoronto.ca

Abstract

We investigate batch composition in semi-online Direct Preference Optimization (DPO) by introducing a fresh data fraction α that controls the proportion of each training batch sampled on-policy versus drawn from a fixed offline buffer. Through a grid search over sync period $s \in \{1, 10, \infty\}$ and $\alpha \in \{0, 0.5, 1.0\}$ on GSM8K with Qwen2.5-1.5B-Instruct, we find that pure offline training overfits on the fixed buffer while a mixed composition ($\alpha = 0.5$) consistently outperforms both pure offline and fully on-policy training at half the generation cost. We also confirm that semi-online DPO ($s = 10$) matches fully online DPO ($s = 1$) across all α values, indicating that batch composition has a larger effect on performance than sync frequency. Our results suggest that a mixture of fresh and offline data offers a more compute-efficient alternative to fully on-policy generation. Code available at <https://github.com/Issey-Sone/DPO-On-Policy-Final-Project>.

1 Introduction

Recent developments in large language models (LLMs) have shown that post-training alignment is critical for allowing pretrained models to follow human intent and follow specific downstream tasks. The benefits of aligning language models has been well established for a variety of tasks [16, 2]. A popularized method for aligning these language models is through Reinforcement Learning (RL) and Proximal Policy Optimization (PPO) to optimize for human preferences or verifiable rewards. For open-ended tasks which are non-verifiable, a reward model trained on human preference rankings is used to estimate response quality, reducing reliance on costly human annotation during training. For less open-ended tasks which are verifiable, such as math and code, the correctness of the answer can be directly checked against a known reference, providing a binary reward signal without the need for a learned reward model.

For the optimization method, PPO comes with a high computational cost, and is very sensitive to hyperparameters which makes it tricky to utilize. Several cheaper, and offline methods have been proposed. With the added assumption of having preference data, Direct Preference Optimization (DPO) [19] reduces the problem to a supervised learning task, it directly increases the likelihood of preferred responses to rejected ones via. log-ratio objective. DPO has also shown results for use with verifiable rewards or with reward models [17]. DPO can also be used in a semi-online manner by periodically synchronizing the generator model weights with the training model, allowing responses to be sampled from a progressively updated policy rather than a fixed one.

Naturally, by modifying the objective function, several variants of DPO have been proposed, including IPO, ORPO, KTO, and SimPO[1, 6, 9, 15]. These methods fall under the broader class of offline preference optimization approaches. However, empirical studies have shown that offline approaches fall behind online methods in a wide variety of benchmarks[10, 22]. Another caveat comes from

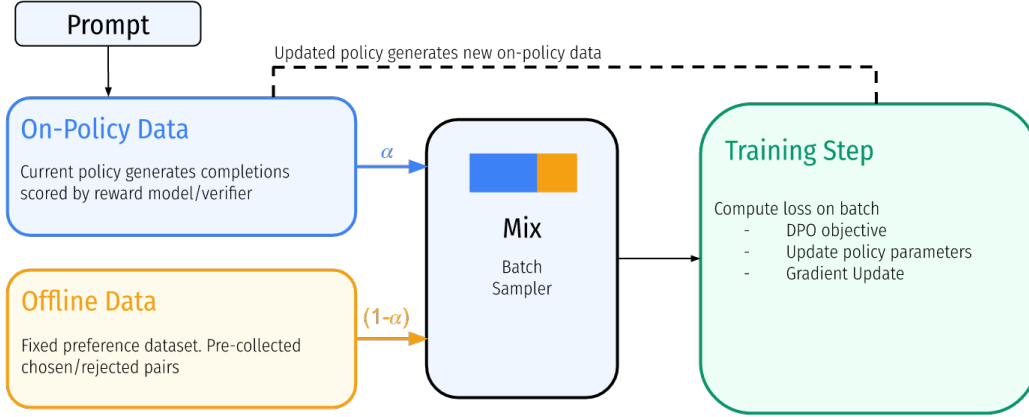


Figure 1: **Semi-online DPO training pipeline.** At each step, on-policy data generated from the current model and offline preference data are combined using a mixing coefficient α . The resulting batch is used to compute the DPO loss and update the policy parameters.

offline methods such as DPO failing to learn consistent preference rankings compared to online approaches. This stems from their pairwise objective, which cannot correct ranking errors inherited from the reference model, leading to a significant alignment gap [3].

More work was sought after to understand the gap between offline and online alignment methods. Experiments have systematically shown that online methods are a Pareto improvement over offline methods across different KL budgets, and persists regardless of loss function or scale[20]. Offline-trained policies become strong at pairwise classification but poor at generation, while online-trained policies show a contrasting pattern. This became evidence of on-policy sampling playing a pivotal role for policy training rather than data quality or coverage[20]. Approaches to narrow this gap include iterative DPO, which regenerates preference pairs from the updated model at the end of each epoch [21, 23, 4], and fully online DPO variants that synchronize generation with training at every step [7, 18]. These ideas were bridged together by introducing a sync period s that controls how frequently the generator model is updated, showing that semi-online DPO ($s = 10$) nearly matches fully online performance at reduced cost[13].

Whilst this approach showed successful results for both verifiable, and non-verifiable tasks across various benchmarks, the framework treats each training batch as coming from either the generator or a fixed offline dataset. However, there isn't anything that prevents a training batch that comes from both. Our method introduces a second knob, the fresh data fraction $\alpha \in [0, 1]$, which controls what proportion of each training batch is sampled on-policy from the generator versus drawn from a pre-collected offline preference buffer as seen in Figure 1. Our experiment aims to answer a simple question: What fraction of fresh samples is actually necessary? Specifically, we want to see how each fraction of samples performs for verifiable tasks.

Based on our experimental results, our contributions are as follows. First, we show that pure offline DPO plateaus early as the model overfits on the fixed preference buffer, with training loss approaching zero while test accuracy stagnates, confirming that data diversity, not just optimization, is a bottleneck in offline training. Second, we find that a mixed regime was outperforming pure offline and online training across all sync periods, suggesting that the offline pairs provide stable gradient signal, whilst fresh pairs act as regularization. Third, we reproduce the findings of [13] that semi-online DPO performs comparably to fully online DPO, and show this equivalence holds across all mixing values. Finally, we identify $\alpha = 0.5$ as the Pareto-optimal operating point, achieving the highest accuracy per unit of generation cost and roughly doubling the compute efficiency of fully online training. These findings suggest that the common assumption of needing entirely on-policy data for effective preference optimization may be unnecessarily costly, and that a carefully composed mixture of fresh and offline data offers a more practical path to strong post-training performance.

2 Direct Preference Optimization (DPO)

The basis of our work revolves around Direct Preference Optimization [19] for aligning language models. DPO is an offline alignment algorithm which is derived from traditional RLHF and PPO [16, 25]. Assuming we have preference data $y_w \succ y_l$, [19] showed that RLHF can be rewritten into a supervised learning objective. The DPO loss is as follows:

$$\mathcal{L}_{\text{DPO}} = \log \sigma \left(\beta \log \frac{\pi(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \quad (1)$$

However, the entire formulation of DPO is entirely offline and the preference pairs are fixed in the beginning. The model trains on responses generated by a potentially different policy, leading to a distribution mismatch that grows as training progresses. Naturally, this mismatch motivates the use of semi-online and online variants of DPO, where preference pairs are periodically or continuously regenerated from an evolving policy.

3 Online and Semi-online DPO

DPO was designed on a purely offline setup where we can generate preference pairs, and train with the DPO loss on the pre-generated responses. Iterative DPO [17] allows multiple iterations of DPO where training occurs over the entire dataset, but we generate a new set of responses using the model from the previous iteration. This offers performance gains over offline DPO by reducing the distribution mismatch between the generating policy and training policy [21, 23, 4]. Semi-online DPO[13] adds a sync period s , which determines how many gradient steps occur between each synchronization of the generator weights with the current policy. Decreasing s makes training more on-policy, with $s = 1$ corresponding to fully online DPO where each batch is generated from the latest model.

In our work, we consider a framework by introducing the fresh data fraction α which controls how much of each batch comes from the generator versus a fixed offline buffer with pre-generated responses. The batch of data is as follows:

$$D_{\text{batch}} = \alpha D_{\text{fresh}} + (1 - \alpha) D_{\text{offline}} \quad (2)$$

The advantage of controlling the amount of each batch, comes from the observation that fresh and offline pairs serve complementary roles during training. Offline pairs provide a stable and consistent gradient signal since it remains fixed throughout training. Fresh pairs, sampled from the current or recently synced policy, introduces diversity which prevents overfitting to the pre-generated pairs. By tuning α , we can balance these two properties, and reduce computational cost by not generating an entirely fresh batch at each sync period.

4 Experimental Setup

We study the effectiveness of our post-training method across two axes: the sync period s , which controls how frequently the generator weights are updated, and the fresh data fraction α , which controls the proportion of fresh data of each training batch. All experiments use a verifiable math task with binary rewards.

4.1 Experiment Configurations

We are analyzing two knobs at the same time, to see if we really need a full online sample to get the best performance out of semi-online DPO. For the task, we compare offline DPO ($s = \infty$), online DPO ($s = 1$), and semi-online DPO ($s = 10$). Along with the sync period, we vary the fresh data fraction across three values ($\alpha \in \{0, 0.5, 1\}$). Respectively, these correspond to pure offline batch, an equal mix of offline and fresh pairs, and fully on-policy batches respectively. In total, this yields a grid of nine configurations that spans the full spectrum from offline to online training in both axes of our training method.

4.2 Hyperparameters, and Training implementation

We initialize model parameters using the Qwen2.5-1.5B-Instruct with LoRA adaptation applied to the query, key, value, and output projection layers. The reference model is kept frozen throughout

training, while the generator model is synchronized with the training model every s steps. At each training step, we sample $\lceil \alpha \cdot B \rceil$ fresh pairs, and the remaining pairs from a pre-constructed offline buffer of 60 valid preference pairs, built from a seed model of 500 benchmark training prompts. For each prompt, we sample 8 responses and form a preference pair by selecting a correct response as the chosen and an incorrect response as the rejected. All experiments are run on a single NVIDIA A100-40GB GPU using bfloat16 precision, with each configuration trained for 150 steps. Full hyperparameter details are provided in Table 2.

4.3 Verifiable Math Problem Task

For our verifiable task, we use the GSM8K dataset [5], a widely used benchmark consisting of 7,473 training and 1,319 test grade-school math problems requiring multi-step reasoning. Like other experiments done in [17, 12], the setup follows a verifiable reward paradigm used in reinforcement learning for mathematical reasoning. Similar to [13], we used Math-Verify [11] to extract the final numeric answer from the model’s response and compare it with the reference answer with a small tolerance. Using this, we can use a binary reward setup similar to [13]. Prompts are formatted using the model’s chat template, with instructions to solve the problem concisely and end with a standardized answer format. We evaluate on a random subset of 150 GSM8K test problems, and average accuracy over 2 random seeds. More details and examples can be seen in appendix section A.

5 Results

Effect of having Fresh Data To study the effect of having fresh data, we examine the pure offline setting ($\alpha = 0$), where all preference pairs are generated prior to training. Naturally, this serves as a baseline and mirrors the standard DPO setup. We then compare against configurations where a portion of each batch is replaced with freshly generated preference pairs from the current or recently synced policy. If the offline buffer provides a sufficient learning signal, then $\alpha = 0$ will perform comparably or better than $\alpha > 0$. However, with a batch size of 4, and 150 training steps, with an online buffer with 60 pairs, the $\alpha = 0$ configuration draws 600 pairs with replacement from only 60 unique examples, meaning that each pair is seen roughly 10 times on average. This setup causes concern for overfitting to the pre-generated pairs, and the diversity introduced by fresh on-policy samples may be important.

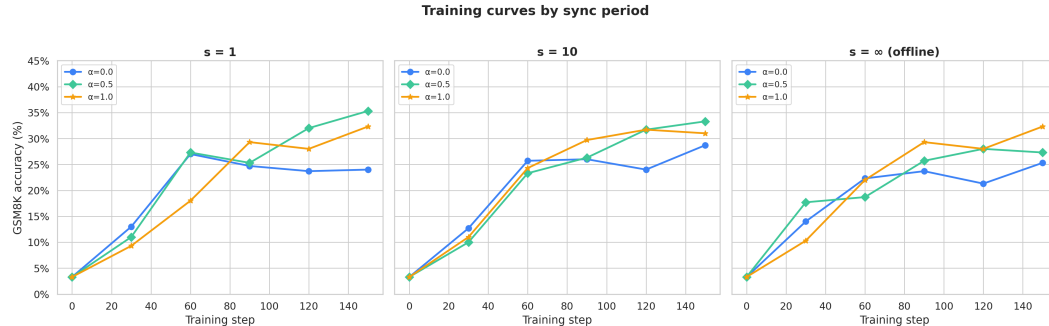


Figure 2: **Training curves across sync periods.** GSM8K validation accuracy is plotted over training steps for different mixing ratios α and sync periods s . Smaller s (more frequent synchronization) corresponds to more on-policy data, while $s = \infty$ represents the offline setting.

Figure 2 shows the training curves across all 9 configurations. As suspected, for the $\alpha = 0$ configurations, the DPO loss drops steadily through training, reaching as low as 0.18 by step 120 in the $s = 1$ configuration (Figure 3). However, the validation accuracy plateaus around 24–29% after step 60 and fails to improve further. The $\alpha > 0$ configurations maintain higher and fluctuating loss throughout training, with the loss typically between 0.4 to 0.8 (Figure 3), however the accuracy steadily rises and doesn’t seem to plateau at step 150. The fresher pairs, although noisier, provide a learning signal that prevent the model from overfitting to the pre-generated pairs.

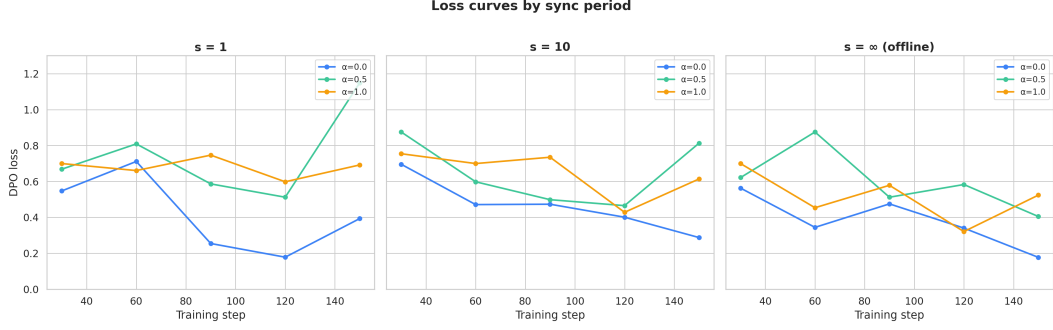


Figure 3: **Loss curves across sync periods.** DPO loss is plotted over training steps for different mixing ratios α and sync periods s . Smaller s corresponds to more frequent synchronization (more on-policy data), while $s = \infty$ denotes the offline setting.

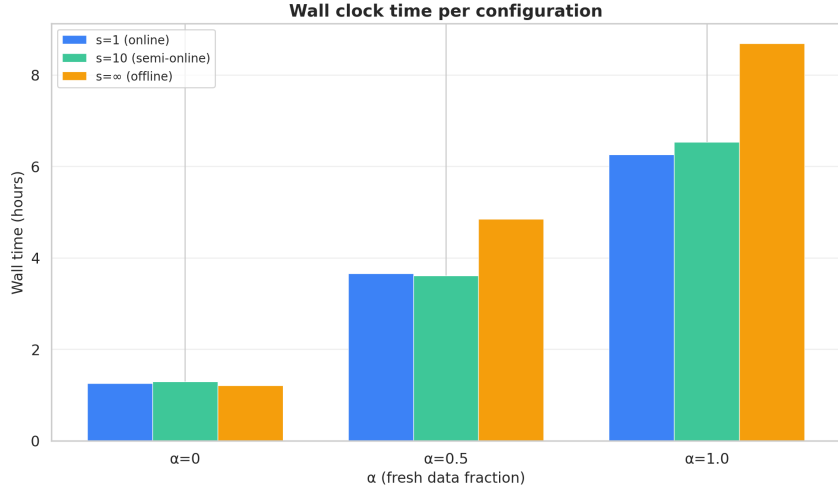


Figure 4: **Wall-clock training time across configurations.** Training time (hours) is shown for different mixing ratios α and sync periods s . Larger α increases cost due to on-policy data generation, while offline training ($s = \infty$) is most expensive overall.

Effect of On-Policy Data Ratio Having established that fresh data prevents overfitting, we now move on to asking: how much fresh data is actually needed? Full on-policy training done in [13, 3, 17] requires generating new preference pairs in the batch at each training step. Looking at the wall time, when $\alpha = 1$ the wall time is consistently the highest between 6–9 hours to complete compared to the roughly 1–2 hours for $\alpha = 0$ (Figure 4). If a smaller fraction of fresh data can achieve comparable accuracy, the computational savings on a small scale would be significant. For this, we compare the best validation accuracy achieved by each α configuration, alongside the total cost generation during training. If fresh data has diminishing returns, the gap between $\alpha = 0.5$ and $\alpha = 1$ would be smaller than the gap between $\alpha = 0$ and $\alpha = 0.5$. In Figure 4, it’s important to note that across various values of α , the wall time is similar between $s \in [1, 10, \infty]$. In [13], with $s = 10$, the generator can work ahead, take the weights once, and generates responses for the next 10 batches in parallel, and sends it back whilst the trainer continues updating. In a such a setup, the sync period s will exhibit advantages in wall time. In our experiment which uses a single GPU, doing everything sequentially, this advantage will disappear. With $s = 1$, the trainer must pause, send its weights to the generator cluster, wait for new generations, then resume. In short, the generator runs asynchronously across the s steps instead of being a synchronous bottleneck. On a single GPU setup, the generation and training happen sequentially on the same GPU, the total work is identical regardless of s . Having similar wall-times across various values of s in our experiment is what we would expect.

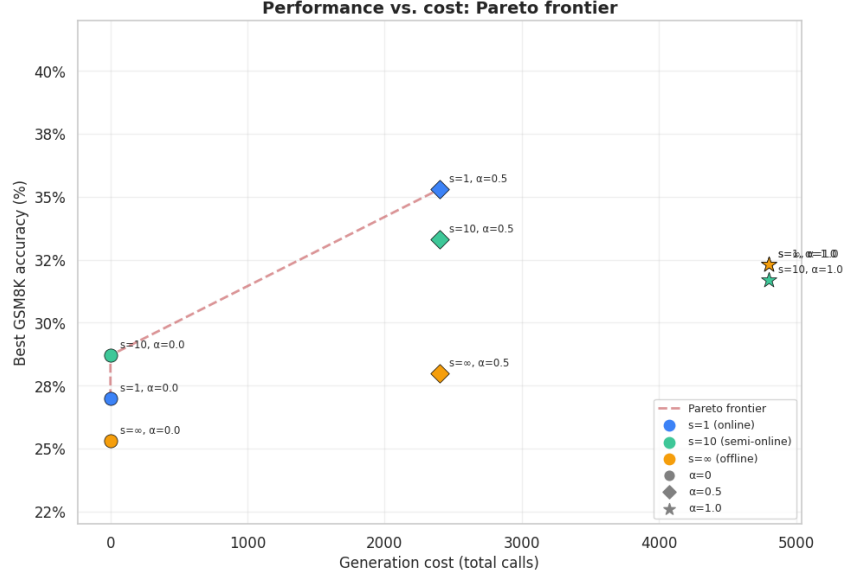


Figure 5: **Performance–cost trade-off across training regimes.** Best GSM8K accuracy is plotted against generation cost for different sync periods s and mixing ratios α . The Pareto frontier highlights configurations that achieve the best accuracy for a given cost.

Table 1: **Best GSM8K accuracy by configuration.**

Training Method	α	Best Acc. (%)	Gen. Cost	Wall Time (hr)
Seed (Qwen2.5-1.5B-Instruct)	–	3.3	0	–
Online DPO ($s = 1$)	0.0	27.0	0	1.3
	0.5	35.3	2,400	3.7
	1.0	32.3	4,800	6.3
Semi-online DPO ($s = 10$)	0.0	28.7	0	1.3
	0.5	33.3	2,400	3.6
	1.0	31.7	4,800	6.5
Offline DPO ($s = \infty$)	0.0	25.3	0	1.2
	0.5	28.0	2,400	4.84
	1.0	32.3	4,800	8.69

Table 1 summarizes the full results of the 9 configurations in our experiment. Across both $s = 1$ and $s = 10$, the $\alpha = 0.5$ configurations achieve the highest or near-highest validation accuracy of 35.3% and 33.3% respectively. Whilst $\alpha = 1.0$ reaches 32.3% and 31.7%. The $\alpha = 0.5$ wall time and generation cost are half of $\alpha = 1.0$ which shows promising results (Table 1). Figure 5 shows the Pareto frontier over all 9 configurations, plotting the best validation accuracy against total generation cost. The two configurations that lie on the frontier are $s = 10, \alpha = 0.0$ with 0 generation cost, and $s = 1, \alpha = 0.5$ with 2,400 generation calls. Notably, the frontier exhibits a steep initial slope between cost 0 and 2,400, followed by a flat or negative return beyond that point. This indicates strong diminishing returns from fresh data generation: the first 50% of fresh data in each batch provides the majority of the accuracy gain, while the second 50% adds compute cost without improving performance. For practitioners operating under compute constraints, this suggests that a mixed batch with $\alpha = 0.5$ at any reasonable sync period offers the best return on generation investment.

Surprisingly, we observe a clear upward trend within the offline setting $s = \infty$, with validation accuracy rising from 25.3% at $\alpha = 0$ to 28.0% at $\alpha = 0.5$, and to 32.3% at $\alpha = 1.0$. Since the generator is never synchronized, fresh pairs are drawn from the same seed policy as the offline buffer, so this improvement is not attributed to on-policy learning. This issue is likely due to the

data diversity, the $\alpha = 0$ configuration will keep resampling the same 60 pairs, whilst the $\alpha > 0$ configurations will generate new responses to randomly selected prompts at each step, even though these responses come from the same underlying policy. This suggests that the offline plateau is driven not only by distribution mismatch between the generating and training policies, but also by the limited diversity of a small fixed buffer.

Effect of sync period Finally, we examine the effect of the sync period s across all values of α . Consistent with the findings of [13], semi-online DPO ($s = 10$) performs comparably to fully online DPO ($s = 1$). The accuracy difference between these two configurations never exceeds 2 percentage point at any α value (Table 1). Based on our experiments, α seems to account for a larger share of the variation in accuracy than s . Moving $\alpha = 0$ to $\alpha = 0.5$ at a fixed sync period yields a 4-8% improvement compared to the 2% moving s . Suggesting that for verifiable tasks at this scale, the composition of each training batch matters more than how frequently the generator is updated.

6 Discussion

6.1 Interpretation of $\alpha = 0.5$

Our results suggest that fresh and offline data serve complementary roles in DPO training. As mentioned earlier, offline pairs provide a stable, smooth gradient signal that makes early training efficient, with all configurations showing similar accuracy gains through step 60, regardless of α . Fresh pairs prevent the language model from memorizing the fixed buffer, which becomes critical later as the number of training steps increase. In our experiment we see the $\alpha = 0.5$ configurations outperforming the $\alpha = 1$ configuration, suggesting that full on-policy batches becomes too noisy during training. The mixed set of offline and fresh pairs balances stability from the offline component, and exploration from the fresh component.

6.2 Limitations

Scale gap Our experiment had several limitations which need to be considered. There is a significant scale gap compared to related work done in [13, 20, 23]. For instance, our Qwen2.5-1.5B-Instruct model on GSM8K is on a considerably smaller scale than the Llama-3.1-8B-Instruct model, and NuminaMath[14] dataset used by [13]. Our models were also fine-tuned with LoRA compared to full supervised fine-tuning done in related work. The LoRA adaptation with rank 32, updates only 56% of the model’s parameters. While this allows efficient weight syncing for the reference and generator model states, it limits the model’s capacity to adapt compared to full fine-tuning. The accuracy differences between configurations may be compressed by this bottleneck, whereas full fine-tuning may reveal larger separations between α values that LoRA may be obscuring. However, full fine-tuning would require maintaining three complete copies of the model weights (training, reference, and generator), which was not feasible within the single-GPU setup. As mentioned earlier, with a single-GPU setup we aren’t able to capture the asynchronous generation benefits that make semi-online training done in [13] attractive at scale.

Offline dataset size With an offline dataset with only 60 preference pairs, this amplifies the overfitting we saw for the $\alpha = 0$ configurations. As mentioned earlier, with a 150 step size and batch size of 4, the $\alpha = 0$ configurations saw each pair roughly 10 times over. This amplifies the overfitting effect and likely overstates the advantage of fresh data compared to a setting with a larger buffer. However, the observation that training loss can decrease while validation accuracy stagnates reflects a real issue of offline DPO that have been seen in other work [20]. A natural extension to this would be to vary the buffer size alongside α to understand how much of the fresh data benefit comes from preventing overfitting buffer memorization versus true on-policy signal. With a larger offline preference pair size, we would expect to see the $\alpha = 0$ plateau we observed occurring later in training but eventually emerge if we push the number of training steps. The $\alpha > 0$ configurations would likely provide the benefits of on-policy data, even when data diversity is no longer a bottleneck.

Evaluation stage The evaluation size was 150 randomly sampled GSM8K test problems averaged over 2 seeds, yielding a standard error of approximated 3-4%. This means that the small accuracy differences we saw in our experiment would not be considered statistically significant. A more robust

evaluation would be to use a larger GSM8K test set with 5 or more seeds, however, this would require higher computational needs. The training and evaluation prompts being drawn from GSM8K, means that we are only measuring in-distribution performance only. The model’s ability to generalize to other mathematical benchmarks such as MATH [8] or NuminaMath[14] remains to be seen. It would be worth investigating the benefits of fresh data for out-of-distribution tasks, where memorization of an offline dataset is likely less useful.

Singular task Finally, our experiment is limited to a single verifiable task with binary rewards. As mentioned earlier, evaluating on a diverse set of benchmarks would provide stronger evidence for generalizing our findings. Harder benchmarks would also produce a more natural distribution of correct and incorrect responses during preference pair construction, potentially alleviating the size limitation we saw in our experiment. It’s also important to point out that the work done in [13] goes into both verifiable tasks with rule-based rewards and non-verifiable tasks with learned reward models, and show that the two settings can vary during training. For example, non-verifiable tasks showed less entropy collapse and a tendency toward length hacking due to reward model biases. The effect of our α knob may differ in a non-verifiable setting, where the quality of fresh preference pairs depends on the reward model’s accuracy rather than a deterministic verifier. A reward model could assign inconsistent scores to similar responses, making fresh pairs noisier than in the verifiable case, which could either increase or decrease the optimal α . Extending our analysis to non-verifiable tasks such as instruction following on WildChat [24] with a learned reward model like Athene-RM-8B would test whether the $\alpha = 0.5$ sweet spot generalizes beyond the binary reward setting.

7 Conclusion

We investigated the effect of batch composition on semi-online DPO by introducing a fresh data fraction α that controls how much of each training batch comes from on-policy generation versus a fixed offline buffer. Through a grid search over $\alpha \in \{0, 0.5, 1\}$ and sync period $s \in \{1, 10, \infty\}$ on GSM8K with Qwen2.5-1.5B-Instruct, we find three key results. First, pure offline training $\alpha = 0$ overfits on the offline preference set, with training loss approaching zero while validation accuracy plateaus, confirming that data diversity is a bottleneck in offline DPO. Second, a mixed batch with $\alpha = 0.5$ consistently achieves the highest accuracy across sync periods while incurring half the generation cost of full on-policy training ($\alpha = 1.0$). Lastly, semi-online DPO ($s = 10$) performs comparably to fully online DPO ($s = 1$) across all values of α , reproducing the findings of [13] and demonstrating that batch composition has a larger effect of accuracy than sync frequency. Taken together, our results suggest that the common assumption of needing entirely on-policy data for effective preference optimization is unnecessarily costly, and that a carefully composed mixture of fresh and offline data offers a more compute-efficient path to strong post-training performance.

Future Directions With additional compute, several extensions would strengthen these findings. A larger offline set of 200-300 preference pairs would reduce the overfitting confound and isolate the effect of policy freshness from data diversity. Going through finer values of α , particularly $\alpha \in \{0.25, 0.35, 0.5, 0.65, 0.75\}$ would show an overall more complete picture of the accuracy-cost tradeoff. Scaling to a larger model such as Llama-3.1-8B-Instruct on more benchmarks would strengthen our findings and claims. An interesting direction would be to implement an adaptive α scheduler that varies the fresh data fraction over the course of training. For instance, starting with a low value of α to take advantage of the stable gradient signal from the pre-generated preference pairs during early stages of training, then increasing α to prevent the model from overfitting. The converse can also be tested as well, start with a high α value to explore, and shift it lower for stabler convergence. Lastly, as mentioned before, extending our work to non-verifiable tasks with learned reward models, and to a joint training setting combining both verifiable and non-verifiable rewards as explored in [13], would determine whether the α setting interacts differently in these circumstances.

Acknowledgments and Disclosure of Funding

I would like to thank Professor Qiang Sun for his guidance and support throughout the course. His lectures and feedback were instrumental in shaping this project.

References

- [1] Mohammad Gheshlaghi Azar, Mark Rowland, Bilal Piot, Daniel Guo, Daniele Calandriello, Michal Valko, and Rémi Munos. A general theoretical paradigm to understand learning from human preferences, 2023.
- [2] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, Nicholas Joseph, Saurav Kadavath, Jackson Kernion, Tom Conerly, Sheer El-Showk, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Tristan Hume, Scott Johnston, Shauna Kravec, Liane Lovitt, Neel Nanda, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Sam McCandlish, Chris Olah, Ben Mann, and Jared Kaplan. Training a helpful and harmless assistant with reinforcement learning from human feedback, 2022.
- [3] Angelica Chen, Sadhika Malladi, Lily H. Zhang, Xinyi Chen, Qiuyi Zhang, Rajesh Ranganath, and Kyunghyun Cho. Preference learning algorithms do not learn preference rankings. *arXiv preprint arXiv:2405.19534*, 2024.
- [4] Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning converts weak language models to strong language models, 2024.
- [5] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021.
- [6] Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. Kto: Model alignment as prospect theoretic optimization, 2024.
- [7] Shangmin Guo, Biao Zhang, Tianlin Liu, Tianqi Liu, Misha Khalman, Felipe Llinares, Alexandre Rame, Thomas Mesnard, Yao Zhao, Bilal Piot, Johan Ferret, and Mathieu Blondel. Direct language model alignment from online ai feedback, 2024.
- [8] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset, 2021.
- [9] Jiwoo Hong, Noah Lee, and James Thorne. Orpo: Monolithic preference optimization without reference model, 2024.
- [10] Hamish Ivison, Yizhong Wang, Jiacheng Liu, Zeqiu Wu, Valentina Pyatkin, Nathan Lambert, Noah A. Smith, Yejin Choi, and Hannaneh Hajishirzi. Unpacking dpo and ppo: Disentangling best practices for learning from preference feedback, 2024.
- [11] Hynek Kydlíček. Math-Verify: Math Verification Library.
- [12] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Oyvind Tafjord, Chris Wilhelm, Luca Soldaini, Noah A. Smith, Yizhong Wang, Pradeep Dasigi, and Hannaneh Hajishirzi. Tulu 3: Pushing frontiers in open language model post-training, 2025.
- [13] Jack Lanchantin, Angelica Chen, Janice Lan, Xian Li, Swarnadeep Saha, Tianlu Wang, Jing Xu, Ping Yu, Weizhe Yuan, Jason E Weston, Sainbayar Sukhbaatar, and Ilia Kulikov. Bridging offline and online reinforcement learning for llms, 2025.
- [14] Jia Li et al. Numinamath: The largest public dataset in ai4maths with 860k pairs of competition math problems and solutions. https://faculty.bicmr.pku.edu.cn/~dongbin/Publications/numina_dataset.pdf, 2024. Technical report.
- [15] Yu Meng, Mengzhou Xia, and Danqi Chen. Simpo: Simple preference optimization with a reference-free reward, 2024.
- [16] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022.

- [17] Richard Yuanzhe Pang, Weizhe Yuan, Kyunghyun Cho, He He, Sainbayar Sukhbaatar, and Jason Weston. Iterative reasoning preference optimization, 2024.
- [18] Biqing Qi, Pengfei Li, Fangyuan Li, Junqi Gao, Kaiyan Zhang, and Bowen Zhou. Online dpo: Online direct preference optimization with fast-slow chasing, 2024.
- [19] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2024.
- [20] Yunhao Tang, Daniel Zhaohan Guo, Zeyu Zheng, Daniele Calandriello, Yuan Cao, Eugene Tarassov, Rémi Munos, Bernardo Ávila Pires, Michal Valko, Yong Cheng, and Will Dabney. Understanding the performance gap between online and offline alignment algorithms, 2024.
- [21] Jing Xu, Andrew Lee, Sainbayar Sukhbaatar, and Jason Weston. Some things are more cringe than others: Iterative preference optimization with the pairwise cringe loss, 2024.
- [22] Shusheng Xu, Wei Fu, Jiaxuan Gao, Wenjie Ye, Weilin Liu, Zhiyu Mei, Guangju Wang, Chao Yu, and Yi Wu. Is dpo superior to ppo for llm alignment? a comprehensive study, 2024.
- [23] Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models, 2025.
- [24] Wenting Zhao, Xiang Ren, Jack Hessel, Claire Cardie, Yejin Choi, and Yuntian Deng. Wildchat: 1m chatgpt interaction logs in the wild, 2024.
- [25] Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences, 2020.

A Training Details

Table 2: **Hyperparameter Settings**

Parameter	Value	Parameter	Value
Model	Qwen2.5-1.5B-Instruct	Batch size	4
LoRA rank r	32	Training steps	150
LoRA α	64	Warmup steps	10
LoRA dropout	0.05	Num. generations	8
KL β	0.1	Max gen. length	384
Learning rate	1e-5	Train prompts	500
Adam ϵ	1e-4	Offline buffer size	60
Gradient clip	1.0	Eval samples	150
Precision	bfloat16	Eval seeds	2

A.1 Prompt

We format all prompts using the Qwen2.5-1.5B-Instruct chat template via the `apply_chat_template` function from the Hugging Face Transformers library. The user message follows a fixed structure requesting a concise solution with a standardized answer format. An example is shown in Figure 6.

A.2 Sample Outputs

We provide example model responses to illustrate the format of correct and incorrect generations used to construct preference pairs.

Prompt Template

```
<|im_start|>system
You are Qwen, created by Alibaba Cloud. You are a helpful
assistant.<|im_end|>
<|im_start|>user
Solve this math problem and be concise. End with 'The answer is
[NUMBER].'

Problem: {problem}
<|im_end|>
<|im_start|>assistant
```

Figure 6: Chat-formatted prompt template used for all GSM8K problems. The system message is automatically prepended by the chat template. The {problem} placeholder is replaced with the GSM8K question.

Example 1: Correct Response (Chosen)

Problem: A store sells apples for \$2 each and oranges for \$3 each. If Sarah buys 4 apples and 5 oranges, how much does she spend in total?

Gold Answer: 23

Model Response:

Apples: $4 \times \$2 = \8

Oranges: $5 \times \$3 = \15

Total: $\$8 + \$15 = \$23$

The answer is 23.

Example 3: Correct Response with Chain-of-Thought

Problem: Tom has 3 times as many marbles as Jerry. Jerry has 12 marbles. Tom gives half of his marbles to Jerry. How many marbles does Jerry have now?

Gold Answer: 30

Model Response:

Jerry has 12 marbles. Tom has $3 \times 12 = 36$ marbles. Tom gives half: $36/2 = 18$ marbles to Jerry. Jerry now has $12 + 18 = 30$ marbles.

The answer is 30.

A.3 Preference Pair Construction

For each prompt, we generate 8 responses using the generator model with temperature 1.0 and top-p 1.0. Each response is verified by extracting the final numeric answer and comparing it to the gold reference. A valid preference pair requires at least one correct and one incorrect response. If all 8 responses are correct or all are incorrect, the prompt is skipped.

A.4 Answer Extraction

We extract the final numeric answer from model responses using a series of regular expression patterns applied in priority order:

1. The (final)? answer is $\$?\boxed{\text{NUMBER}}$ — matches boxed format

Example 2: Incorrect Response (Rejected)

Problem: A store sells apples for \$2 each and oranges for \$3 each. If Sarah buys 4 apples and 5 oranges, how much does she spend in total?

Gold Answer: 23

Model Response:

Apples: $4 \times \$2 = \8

Oranges: $5 \times \$3 = \10

Total: $\$8 + \$10 = \$18$

The answer is 18.

2. The (final)? answer is NUMBER — matches our requested format
3. ##### NUMBER — matches GSM8K gold answer format
4. = NUMBER at end of line — matches trailing equations

If none of these patterns match, the response is treated as incorrect (no answer extracted). Numeric comparison uses a tolerance of 1×10^{-3} to account for floating point representations (e.g., 0.5 matching 0.500).

A.5 Compute Details

Table 3: Compute summary

Training Method	α	Gen. Cost	Wall Time (hr)	Bottleneck
Online DPO ($s=1$)	0.0	0	1.3	Eval only
	0.5	2,400	3.7	Fresh generation
	1.0	4,800	6.3	Fresh generation
Semi-Online DPO ($s=10$)	0.0	0	1.3	Eval only
	0.5	2,400	3.6	Fresh generation
	1.0	4,800	6.5	Fresh generation
Offline DPO ($s=\infty$)	0.0	0	1.3	Eval only
	0.5	2,400	5.0	Fresh generation
	1.0	4,800	9.0	Fresh generation

B Impact Statement

This work studies training methodology for language model alignment and does not introduce new models, datasets, or capabilities. Our findings on compute-efficient batch composition for DPO could reduce the energy and hardware costs associated with post-training language models, making alignment techniques more accessible to researchers with limited computational resources. More broadly, improving the efficiency of preference optimization contributes to the goal of aligning language models with human intent, which is a prerequisite for their safe deployment. However, as with all alignment research, improvements to training methods could be applied to optimize for harmful objectives if the underlying reward signal is misspecified. We do not foresee any direct negative societal consequences specific to this work beyond those inherent to language model development generally.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [NA].
- [NA] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for NA).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will be asked to also include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While "[Yes]" is generally preferable to "[No]", it is perfectly acceptable to answer "[No]" provided a proper justification is given (e.g., "error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering "[No]" or "[NA]" is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”,**
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers.**

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The contributions and scope of the paper are well summarized in the abstract and thorough in the introduction.

Guidelines:

- The answer NA means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The limitations are very clear and well summarized in the discussion section.

Guidelines:

- The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [No]

Justification: There is no theoretical result of guarantees discussed in this paper. Readers may refer to the original paper cited for full theoretical details and proofs.

Guidelines:

- The answer NA means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The experiment details are well discussed in the experimental setup, and more training details are in Appendix A. Code and GitHub page is provided in the abstract.

Guidelines:

- The answer NA means that the paper does not include experiments.
- If the paper includes experiments, a No answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: Code and GitHub page is provided in the abstract. README in the GitHub contains all the information needed to run the experiment.

Guidelines:

- The answer NA means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://nips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so “No” is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://nips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.

- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyper-parameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

Answer: [\[Yes\]](#)

Justification: Training details are in Appendix A.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [\[No\]](#)

Justification: Due to the limited compute setup on a single GPU, it's hard and costly to run repeated experiments.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The authors should answer "Yes" if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).
- If error bars are reported in tables or plots, The authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [\[Yes\]](#)

Justification: Information for compute resources is in Section 4.2.

Guidelines:

- The answer NA means that the paper does not include experiments.

- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The authors have reviewed and followed the NeurIPS Code of Ethics.

Guidelines:

- The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Provided in Appendix section B.

Guidelines:

- The answer NA means that there is no societal impact of the work performed.
- If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

Answer: [No]

Justification: This work does not release any new models or datasets. All experiments use the publicly available Qwen2.5-1.5B-Instruct model and the open-source GSM8K dataset.

Guidelines:

- The answer NA means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [\[Yes\]](#)

Justification: All cited work and assets are in the References.

Guidelines:

- The answer NA means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[NA\]](#)

Justification: There are no new assets introduced in this paper.

Guidelines:

- The answer NA means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [NA]

Justification: There are no crowdsourcing, and research with human subjects here.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [NA]

Justification: There are no crowdsourcing, and research with human subjects here.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does not impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: We evaluate our proposed method on a standard benchmarks and example outputs are provided in Appendix section A.

Guidelines:

- The answer NA means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy (<https://neurips.cc/Conferences/2025/LLM>) for what should or should not be described.